DEVPILOT AI

Java Coding Standards

Practical engineering guidance for Java 21 projects

A retrieval-friendly reference for the Research Agent and Coding Agent. Designed to support consistent implementation, reviewable changes, safe defaults, and maintainable Java code across existing projects.

Document	Value
Version baseline	Java 21 language and standard library
Audience	Java engineers, code reviewers, AI Research and Coding Agents
Primary use	Engineering knowledge-base retrieval and implementation guidance
Normative terms	MUST / SHOULD / MAY
Compatibility note	Inspect the target project's build, style, and runtime constraints before applying rules.

Important: These are general Java engineering standards, not permission to rewrite a project's architecture. Existing conventions and constraints take precedence unless they are unsafe, incorrect, or explicitly being changed.

Heading: Document Navigation — Contents and usage

Use this document as a set of focused engineering reference sections. The literal **Heading:** and **Section:** prefixes are intentionally included in the text so a section-aware RAG chunker can identify topics from extracted PDF text.

1. Scope, language, and rule strength
2. Java 21 conventions and baseline practices
3. Naming conventions
4. Class design, encapsulation, and composition
5. SOLID principles
6. Clean code, cohesion, coupling, and methods
7. Null safety, Optional, and validation
8. Collections, generics, streams, and lambdas
9. Exception handling
10. Logging and sensitive information
11. Immutability, records, enums, and modern Java
12. Resource management
13. Comments, JavaDoc, and self-documenting code
14. Code smells and duplication
15. Concurrency and thread safety
16. Package and directory structure
17. Dependency-selection guidance
18. Requirement implementation workflow
19. Incorrect versus corrected examples
20. Agent-specific implementation rules
21. Code-review checklist

How to apply this guide: First inspect the repository and its conventions. Apply the MUST rules by default, treat SHOULD rules as strong defaults with justified exceptions, and use MAY techniques only when they solve a real problem.

Heading: Scope and Rule Strength

Section: Purpose and audience

This document defines practical Java coding standards for production-oriented Java 21 code. It is intended for engineers and automated agents that inspect, implement, or review changes in an existing repository. It covers code design and implementation practices; it does not replace project-specific architecture documents, security policies, API contracts, or build instructions.

Section: MUST, SHOULD, and MAY

Term	Meaning	Example
MUST	Required unless a documented project-specific constraint justifies an exception.	MUST avoid swallowing exceptions silently.
SHOULD	Strong default; an exception is acceptable when the reason is clear and recorded.	SHOULD prefer composition over inheritance for reusable behavior.
MAY	Optional technique to use when appropriate.	MAY use a record for a simple immutable data carrier.

Section: Repository-first rule

Before editing code, the agent MUST inspect the relevant source files, build file, package layout, tests, and nearby implementations. The existing codebase may already have a naming convention, formatter, static-analysis tool, or domain-specific pattern. Do not impose a new convention on a small change without a reason.

- MUST preserve public APIs and observable behavior unless the task explicitly requires a change.
- MUST make the smallest coherent change that satisfies the requirement.
- SHOULD follow the local style when it is consistent and safe.
- MUST identify assumptions and missing requirements instead of inventing domain behavior.
- MAY propose a broader refactor separately when the requested change exposes a larger design problem.

Heading: Java 21 Conventions

Section: Language and runtime baseline

Java 21 is an LTS release and supports modern language features such as records, sealed classes, pattern matching for `instanceof`, switch expressions, text blocks, and virtual threads. A feature should be used only when it improves clarity or solves a concrete need. Confirm the project's configured Java release and runtime before using a feature; a Java 21 source file cannot compile under an older configured release.

```
// Example Maven compiler baseline (confirm project conventions first)
&lt;properties&gt;
    &lt;maven.compiler.release&gt;21&lt;/maven.compiler.release&gt;
    &lt;project.build.sourceEncoding&gt;UTF-8&lt;/project.build.sourceEncoding&gt;
&lt;/properties&gt;
```

Section: Baseline practices

- MUST keep methods and types readable, explicit, and easy to test.
- MUST avoid deprecated APIs when a supported alternative is available and appropriate.
- SHOULD use local-variable type inference (`var`) only when the initializer makes the type obvious.
- SHOULD use switch expressions when they reduce branching and keep all cases visible.
- MAY use records for immutable data carriers and sealed types when a closed hierarchy is a real domain constraint.
- MUST not adopt preview features merely because the compiler can enable them; preview features introduce build and runtime constraints.

Section: Prefer clear type intent

```
// Clear: the declared type communicates the abstraction.
List&lt;String&gt; names = loadNames();

// Also reasonable when the initializer makes the type obvious.
var activeUsers = userRepository.findActiveUsers();

// Avoid when the type is not obvious from the initializer.
var result = process(input);
```

Use `var` for local variables only. It does not make Java dynamically typed: the compiler still infers a static type. Avoid it when it hides a meaningful interface or makes a complex expression harder to understand.

Heading: Naming Conventions

Section: Standard names

Element	Convention	Example
Class / record / enum	UpperCamelCase; noun or noun phrase	InvoiceService, PaymentStatus
Interface	UpperCamelCase; role or capability	PaymentGateway, ClockProvider
Method	lowerCamelCase; verb or verb phrase	calculateTotal(), findById()
Variable / parameter	lowerCamelCase; meaningful noun	retryCount, customerId
Constant	UPPER_SNAKE_CASE for static final constants	DEFAULT_TIMEOUT
Package	lowercase; short, stable, domain-oriented	com.example.billing
Type parameter	Short conventional name or descriptive name	T, ID, RequestT
Boolean	Question/state wording	isActive, hasPermission, canRetry

Section: Naming rules

- MUST choose names that reveal intent rather than implementation trivia.
- MUST avoid cryptic abbreviations unless they are established domain terms (for example, URL or SKU).
- SHOULD distinguish units in names where confusion is plausible, such as timeoutMillis or amountInCents.
- SHOULD use domain language consistently; do not alternate between customer, client, and account for the same concept without a domain reason.
- MUST avoid misleading names, especially for collections, booleans, asynchronous work, and methods with side effects.
- MAY use a short loop index such as i when its scope is tiny and conventional.

Section: Examples

```
// Weak names
int d;
List&lt;User&gt; data;
boolean flag;
void handle(List&lt;User&gt; x) { ... }

// More descriptive
int retryDelaySeconds;
List&lt;User&gt; activeUsers;
boolean hasWriteAccess;
void notifyInactiveUsers(List&lt;User&gt; inactiveUsers) { ... }
```

A name should not encode information that can change independently, such as a developer's initials or a temporary ticket number. Prefer stable concepts and meaningful intent.

Heading: Class Design and Object-Oriented Boundaries

Section: Encapsulation and invariants

Encapsulation means a type controls access to its internal state and preserves its own invariants. A class should not expose mutable fields or return internal mutable collections when callers could corrupt its state. Validate values at the boundary where the class can best enforce its rules.

```
public final class Discount {
    private final BigDecimal percentage;

    public Discount(BigDecimal percentage) {
        Objects.requireNonNull(percentage, "percentage must not be null");
        if (percentage.signum() &lt; 0 || percentage.compareTo(new BigDecimal("100")) &gt; 0) {
            throw new IllegalArgumentException("percentage must be between 0 and 100");
        }
        this.percentage = percentage;
    }

    public BigDecimal percentage() {
        return percentage;
    }
}
```

Section: Abstraction

An abstraction exposes the operations a caller needs and hides details that should be free to change. Do not create interfaces automatically for every class. Introduce an interface when it expresses a meaningful capability, enables a real substitution boundary, or isolates an external dependency.

Section: Inheritance versus composition

Inheritance models an “is-a” relationship and creates coupling to a base type's behavior and lifecycle. Composition models “has-a” or “uses” relationships and is usually more flexible for assembling behavior.

```
// Prefer composition when behavior can be delegated.
public final class ReportService {
    private final ReportFormatter formatter;

    public ReportService(ReportFormatter formatter) {
        this.formatter = Objects.requireNonNull(formatter);
    }

    public String createReport(Report report) {
        return formatter.format(report);
    }
}
```

- MUST keep fields private unless a specific language or framework requirement dictates otherwise.
- SHOULD prefer final classes when extension is not part of the design contract.
- SHOULD favor composition over inheritance for code reuse.
- MUST not call overridable methods from constructors; subclass state may not yet be initialized.
- MAY use inheritance when the subtype truly satisfies the base type's contract and the hierarchy is stable.

Heading: SOLID Principles

Section: Single Responsibility Principle

A class should have one cohesive responsibility, meaning one primary reason to change. It does not mean every method needs a separate class. Separate responsibilities when their rules, owners, or change patterns differ.

```
// Avoid a class that validates orders, writes SQL, sends email, and formats PDFs.
// Prefer a cohesive service that coordinates focused collaborators.
public final class OrderService {
    private final OrderRepository repository;
    private final ReceiptSender receiptSender;

    public OrderService(OrderRepository repository, ReceiptSender receiptSender) {
```

```
        this.repository = repository;
        this.receiptSender = receiptSender;
    }

    public Order placeOrder(CreateOrder command) {
        Order order = Order.from(command);
        Order saved = repository.save(order);
        receiptSender.sendReceipt(saved);
        return saved;
    }
}
```

Section: Open/Closed Principle

A module should be open to extension but closed to repeated modification for every new variant. Use polymorphism, a strategy, or a data-driven mapping when new variants are expected. Do not build an elaborate extension framework for a stable two-case branch.

```
public interface ShippingCostPolicy {
    Money calculate(Shipment shipment);
}

public final class StandardShippingPolicy implements ShippingCostPolicy {
    @Override
    public Money calculate(Shipment shipment) {
        return shipment.baseCost();
    }
}

public final class ExpressShippingPolicy implements ShippingCostPolicy {
    @Override
    public Money calculate(Shipment shipment) {
        return shipment.baseCost().plus(shipment.expressSurcharge());
    }
}
```

Section: Liskov Substitution Principle

A subtype must honor the behavioral contract of its supertype. It should not unexpectedly strengthen preconditions, weaken guarantees, or change semantics that callers rely on. If a subtype cannot safely satisfy the contract, use a different abstraction.

Section: Interface Segregation Principle

Clients should not be forced to depend on methods they do not use. Prefer small capability interfaces over a “god interface” that combines unrelated operations. Keep interfaces cohesive rather than making them small for the sake of a count.

Section: Dependency Inversion Principle

High-level business rules should not depend directly on low-level infrastructure details. Both should depend on stable abstractions where a real boundary is needed. Constructor injection makes those dependencies visible and easy to replace in tests.

> **Practical SOLID test:** If a class is difficult to test without starting unrelated infrastructure, has many unrelated reasons to change, or requires editing a central conditional every time a new behavior is added, examine its boundaries before adding more code.

Heading: Clean Code and Method Design

Section: Cohesion and coupling

High cohesion means a module's contents work together toward one purpose. Low coupling means modules rely on few, stable details of each other. Aim for clear boundaries, not the theoretical elimination of all dependencies: useful software necessarily collaborates across components.

Section: Method size and control flow

- SHOULD keep each method focused on one level of abstraction.

- SHOULD use guard clauses to handle invalid or exceptional preconditions early when this improves readability.
- MUST make side effects visible in method names or documented contracts.
- SHOULD avoid boolean parameters when they make a call ambiguous; named alternatives or an options type may be clearer.
- MUST avoid excessive nesting where early returns or extracted methods communicate intent better.
- MAY keep a longer method when its linear flow is genuinely clearer than fragmented helper methods.

```
// Ambiguous: what does true mean?
createInvoice(order, true);

// Clearer alternatives
createInvoice(order, InvoiceDelivery.EMAIL);
createAndEmailInvoice(order);
```

Section: Avoid unnecessary abstraction

A helper method is useful when it names a meaningful operation, removes meaningful duplication, or isolates complexity. Extracting a one-line expression into a method with no semantic name can make code harder to follow. Do not introduce factories, managers, providers, or generic frameworks without a concrete need.

Section: Prefer explicit domain operations

```
// Less expressive
if (order.getStatus() == 2) {
    ...
}

// More expressive
if (order.status() == OrderStatus.PAID) {
    ...
}
```

Heading: Null Safety, Optional, and Validation

Section: Null-handling policy

Null is a frequent source of runtime failures. Define which values are required, which are optional, and which empty states are valid. Validate required inputs at public boundaries, and avoid repeatedly checking the same invariant deep inside a call chain.

- MUST reject invalid required inputs with a clear exception or validation error.
- MUST not return null for a collection when an empty collection represents “no results.”
- SHOULD make fields non-null by construction where possible.
- SHOULD use Objects.requireNonNull for constructor dependencies and required internal values when no domain-specific validation is more appropriate.
- MUST not use Optional.get() without proving a value is present; prefer map, flatMap, orElse, orElseGet, and explicit handling.
- SHOULD reserve Optional primarily for return values that may legitimately have no result, rather than using it as a field, method parameter, or collection element.

Section: Optional examples

```
// A repository lookup may legitimately find no user.
Optional&lt;User&gt; findUserByEmail(String email);

// Good: handle the absent case deliberately.
User user = findUserByEmail(email)
    .orElseThrow(() -&gt; new UserNotFoundException("User not found"));

// Avoid eager fallback work when it is expensive.
Config config = findConfig()
    .orElseGet(this::loadDefaultConfig);
```

Section: Input validation

Validate external data at system boundaries: HTTP request DTOs, command-line arguments, file imports, message payloads, and public library APIs. Domain objects should still preserve critical invariants so they cannot be created in an invalid state through an alternate path.

```
public record CreateUserRequest(
        @NotBlank String displayName,
        @Email @NotBlank String email
) {}

// Domain-level validation may still be required when constructing a User
// outside the HTTP controller.
public record UserId(UUID value) {
    public UserId {
        Objects.requireNonNull(value, "value must not be null");
    }
}
```

The Jakarta validation annotations above require a validation provider in the application. In Spring Boot applications this is commonly supplied by the validation starter; a plain Java module does not need to add it if validation is handled another way.

Heading: Collections, Generics, Streams, and Lambdas

Section: Choosing a collection

Need	Typical choice	Notes
Unique elements	Set	Choose HashSet for general uniqueness; LinkedHashSet if insertion order matters; TreeSet for sorted order.
Indexed sequence	List	ArrayList is a general default; choose another implementation only for a reason.
Key-to-value lookup	Map	HashMap is a general default; LinkedHashMap preserves insertion order; TreeMap sorts keys.
FIFO work queue	Queue / Deque	ArrayDeque is suitable for many in-memory queue/stack needs.
Concurrent access	ConcurrentHashMap or concurrent queue	Choose based on operations and atomicity requirements; thread safety is not automatic for compound workflows.

Section: Program to interfaces

Declare variables and parameters using the abstraction that callers need, such as List or Map. This reduces accidental dependence on implementation details and makes substitutions easier. Use a concrete type when construction or a concrete capability is relevant.

Section: Generics

```
// Generic code preserves type safety without casts.
public static &lt;T&gt; List&lt;T&gt; nonNullItems(Collection&lt;T&gt; items) {
    Objects.requireNonNull(items, "items must not be null");
    return items.stream()
            .filter(Objects::nonNull)
            .toList();
}
```

Avoid raw types such as `List` without a type parameter. They weaken compile-time checks and can cause runtime ClassCastException. Use bounded type parameters only when the algorithm needs the bound, for example `T extends Comparable<T>`.

Section: Streams and loops

Streams are useful for declarative transformations and aggregations. Traditional loops remain appropriate when they are clearer, involve complex control flow, need early exit, or perform side effects that would make a stream difficult to reason about.

```
// Clear transformation
List&amp;lt;String&amp;gt; activeEmails = users.stream()
        .filter(User::isActive)
        .map(User::email)
        .toList();

// A loop may be clearer for multi-step logic and early exit.
User match = null;
for (User user : users) {
    if (user.isActive() &amp;amp;&amp;amp; matchesPolicy(user)) {
        match = user;
        break;
    }
}
```

- MUST avoid modifying a source collection while traversing it unless the API explicitly supports that operation.
- MUST not rely on stream execution order unless the operation and stream contract preserve it.
- SHOULD avoid side effects inside map, filter, and other intermediate operations.
- SHOULD avoid parallel streams for I/O-bound work or when the common ForkJoinPool could cause contention.
- MAY use immutable results such as Stream.toList() when callers should not mutate the result.

Section: Equality and hash codes

Types used as keys in hash-based collections MUST implement equals and hashCode consistently. Equal objects must have equal hash codes. Avoid mutating fields involved in equality or hashing while an object is used as a key in a HashMap or member of a HashSet.

Heading: Exception Handling

Section: Handle expected failures explicitly

Exceptions should communicate failure conditions, not replace ordinary control flow. Use specific exception types and handle an exception at the layer that can recover, translate it into a useful domain/API error, or add relevant context. Do not catch an exception merely to log it and rethrow it unchanged unless the extra logging is justified and avoids duplicate logging.

```
// Incorrect: failure disappears and callers assume success.
try {
    invoiceRepository.save(invoice);
} catch (Exception ignored) {
}

// Better: let the failure propagate, or translate it at a boundary.
try {
    invoiceRepository.save(invoice);
} catch (DataAccessException ex) {
    throw new InvoicePersistenceException(
            "Could not save invoice " + invoice.id(), ex);
}
```

Section: Custom exceptions

Create a custom exception when it expresses a meaningful domain or boundary failure, enables targeted handling, or carries structured context. Do not create a new exception class for every line of code. Preserve the original cause when translating an exception.

```
public final class InvoiceNotFoundException extends RuntimeException {
    public InvoiceNotFoundException(UUID invoiceId) {
        super("Invoice not found: " + invoiceId);
    }
}

public final class InvoicePersistenceException extends RuntimeException {
    public InvoicePersistenceException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Section: Checked versus unchecked exceptions

Use checked exceptions when callers can reasonably be expected to recover and the API contract benefits from requiring handling. Use unchecked exceptions for programming errors, violated invariants, and many domain/service failures in modern application code. Follow established project conventions and do not convert every exception indiscriminately.

- MUST never swallow an exception silently.
- MUST preserve the cause when wrapping an exception.
- MUST not catch Throwable for routine recovery; it includes serious JVM errors.
- SHOULD catch the narrowest useful exception type.
- SHOULD not expose stack traces or internal implementation details in user-facing messages.
- MAY retry transient failures only when the operation is safe to retry and retry limits/backoff are defined.

Heading: Logging and Sensitive Information

Section: Use levels intentionally

Level	Use	Avoid
ERROR	An operation failed and needs attention or an error response.	Logging every handled validation failure as an application outage.
WARN	Unexpected but recoverable conditions, degraded behavior, or suspicious patterns.	Using WARN for routine control flow.
INFO	Meaningful lifecycle or business events at an appropriate frequency.	Logging every method entry/exit in production.
DEBUG	Diagnostic details useful during investigation.	Assuming debug logs are always private or disabled.
TRACE	Very detailed diagnostics for targeted troubleshooting.	High-volume traces in normal production operation.

Section: Structured and contextual logging

Prefer parameterized logging over string concatenation so the logging framework can avoid unnecessary formatting when a level is disabled. Include identifiers that help correlate an event, but only when they are safe and useful.

```
private static final Logger log =
        LoggerFactory.getLogger(InvoiceService.class);

public Invoice submit(Invoice invoice) {
    log.info("Submitting invoice id={}", invoice.id());
    try {
        return gateway.submit(invoice);
    } catch (GatewayUnavailableException ex) {
        log.warn("Invoice submission failed; invoiceId={}", invoice.id(), ex);
        throw ex;
    }
}
```

Section: Sensitive-data protection

- MUST never log passwords, access tokens, API keys, private keys, session cookies, or full authentication headers.
- MUST assess whether personal data, payment details, health data, or confidential business data belongs in logs.
- SHOULD log stable identifiers rather than entire request or entity objects.
- SHOULD avoid duplicate stack traces when an exception is logged at multiple layers.
- SHOULD ensure untrusted text cannot forge log entries; sanitize or encode line breaks where logs are consumed as plain text.
- MAY use correlation IDs and structured key-value fields when supported by the project's logging setup.

Agent rule: Before adding a log statement, ask: will this value be safe in production logs, and does it help someone diagnose a real issue? If either answer is no, omit or transform it.

Heading: Immutability, Records, Enums, and Modern Java

Section: Immutability

Immutable objects are easier to reason about because their observable state does not change after construction. Prefer final fields, controlled construction, defensive copies for mutable inputs, and no mutating setters where practical. Immutability does not automatically make referenced mutable objects immutable.

```
public final class Team {
    private final String name;
    private final List&lt;String&gt; members;

    public Team(String name, List&lt;String&gt; members) {
        this.name = Objects.requireNonNull(name, "name");
        this.members = List.copyOf(
                Objects.requireNonNull(members, "members"));
    }

    public String name() {
        return name;
    }

    public List&lt;String&gt; members() {
        return members;
    }
}
```

Section: Records

Use a record when a type primarily represents a transparent, fixed set of data components. Records automatically provide accessors, equals, hashCode, and toString. Their component references are final, but deep immutability is not guaranteed when a component references a mutable object.

```
public record Money(BigDecimal amount, Currency currency) {
    public Money {
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(currency, "currency");
        if (amount.scale() &gt; 2) {
            throw new IllegalArgumentException("amount scale must be &lt;= 2");
        }
    }
}
```

The scale rule is only illustrative; real monetary rules depend on the currency and business requirements. Do not copy domain constraints without validating them against the actual application.

Section: Enums and switch expressions

```
public enum OrderStatus {
    DRAFT, SUBMITTED, PAID, CANCELLED
}

public static boolean canBeCancelled(OrderStatus status) {
    return switch (status) {
        case DRAFT, SUBMITTED -&gt; true;
        case PAID, CANCELLED -&gt; false;
    };
}
```

Enums are safer than unexplained integer or string codes for a fixed set of domain values. When an external API or database uses a code, map it explicitly at the boundary rather than spreading raw values throughout the code.

Section: Pattern matching and sealed types

```
if (value instanceof String text &amp;&amp; !text.isBlank()) {
    consume(text);
}
```

Pattern matching can reduce casts and make type checks clearer. Sealed classes or interfaces are useful when a domain intentionally restricts the set of implementations; they are not required for every interface.

Heading: Resource Management

Section: Try-with-resources

Resources such as streams, readers, writers, sockets, and JDBC resources may hold operating-system or database resources. Use try-with-resources for types that implement AutoCloseable so cleanup occurs on both normal and exceptional paths.

```
Path path = Path.of("input.txt");
try (BufferedReader reader = Files.newBufferedReader(path)) {
    return reader.lines().toList();
} catch (IOException ex) {
    throw new UncheckedIOException("Could not read input file: " + path, ex);
}
```

Section: Resource ownership

- MUST close resources that the code opens and owns.
- MUST not close a resource owned by a caller unless the contract explicitly transfers ownership.
- SHOULD use library/framework lifecycle management for managed resources rather than manually closing injected singletons.
- SHOULD set timeouts and define cleanup behavior for network or database operations where supported.
- MUST not rely on finalization or garbage collection as a resource-management strategy.

Section: Avoid unnecessary buffering or loading

Reading an entire file or result set into memory can be appropriate for small bounded inputs. For unbounded or potentially large inputs, process incrementally or use paging. Choose based on expected size and memory limits rather than assuming all data is small.

Heading: Comments, JavaDoc, and Self-Documenting Code

Section: Comments explain why

Good code communicates what it does through names and structure. Comments should explain rationale, constraints, non-obvious trade-offs, external contracts, or surprising behavior. Comments that merely restate the next line become stale and create noise.

```
// Weak: restates the implementation.
count++; // increment count

// Better: explains a non-obvious business constraint.
// The partner API rejects more than 50 IDs per request.
List&lt;List&lt;UUID&gt;&gt; batches = partition(ids, 50);
```

Section: JavaDoc

Use JavaDoc for public APIs, reusable library types, non-obvious contracts, extension points, and behavior callers need to understand. Document parameters, return values, thrown exceptions, nullability expectations, side effects, thread-safety, and preconditions when these are not obvious.

```
/**
 * Calculates the total after applying the supplied discount.
 *
 * @param subtotal non-negative amount before discount
 * @param discount discount policy to apply
 * @return final amount; never {@code null}
 * @throws IllegalArgumentException if subtotal is negative
 */
public Money calculateTotal(Money subtotal, Discount discount) {
    ...
}
```

Section: TODO and FIXME

A TODO should describe the missing work and, where possible, the reason or linked issue. Do not leave vague TODOs such as “fix this later” without context. Before adding one, determine whether the requirement can be completed now and whether the TODO is acceptable in the target project.

Heading: Code Duplication and Common Code Smells

Section: Duplication

Not every repeated line is harmful duplication. Extract shared logic when the repeated behavior has the same meaning and is likely to evolve together. Similar-looking code that represents different domain concepts may be better kept separate.

Section: Common smells and responses

Smell	Why it hurts	Preferred response
Magic numbers/strings	Intent and units are hidden; changes are error-prone.	Use a named constant or enum when it represents a stable concept.
Long method	Multiple decisions and responsibilities are difficult to test.	Extract cohesive operations and name meaningful steps.
God class	Many reasons to change; high coupling.	Separate responsibilities around stable domain boundaries.
Boolean flag argument	Call site hides which behavior is selected.	Use a named method, enum, or options type when clarity improves.
Primitive obsession	Domain rules are scattered around primitive values.	Introduce a value type when it enforces meaningful invariants.
Shotgun surgery	One change requires edits across many unrelated places.	Improve ownership and centralize the rule behind a stable boundary.
Speculative generality	Unused extension points increase complexity.	Implement the current requirement and extend when a real need appears.

Section: Magic values

```
// Weak
if (attempts &gt; 3) {
    wait(5000);
}

// Clearer, with units and intent
private static final int MAX_RETRY_ATTEMPTS = 3;
private static final Duration RETRY_DELAY = Duration.ofSeconds(5);
```

Use constants for values with domain meaning or values that need coordinated change. Do not create a constant for every literal; values such as zero in a simple arithmetic expression often need no name.

Heading: Concurrency and Thread Safety

Section: Shared mutable state

Concurrency bugs arise when threads access shared mutable state without appropriate coordination. Prefer immutable data, local variables, and message passing when practical. If shared state is necessary, document its ownership and synchronization strategy.

- MUST not assume that a normal HashMap, ArrayList, or mutable object is thread-safe.
- MUST not assume that making a reference final makes the referenced object thread-safe.
- SHOULD use high-level concurrency utilities from java.util.concurrent rather than hand-built wait/notify coordination.

- SHOULD minimize synchronized regions and avoid holding locks while performing slow I/O or calling unknown code.
- MUST consider atomicity across multiple operations; a thread-safe collection does not make a multi-step workflow atomic.
- SHOULD write deterministic concurrency tests where practical, avoiding timing assumptions and arbitrary sleeps.

Section: Atomicity and visibility

```
// Not atomic: two threads can both observe the same value and update it.
if (counter &lt; limit) {
    counter++;
}

// For a simple atomic counter:
AtomicInteger counter = new AtomicInteger();
int current = counter.incrementAndGet();
```

AtomicInteger makes individual operations atomic, but it does not automatically make a multi-step invariant atomic. If correctness depends on “check then update” as one indivisible operation, use an appropriate atomic operation, lock, or other coordination strategy.

Section: Virtual threads

Java 21 supports virtual threads for high-concurrency, blocking-style workloads. They can simplify scalable I/O-bound concurrency, but they do not make CPU-bound work faster and do not remove the need for limits on scarce external resources such as database connections. Adopt them only when the runtime and framework integration support the intended execution model.

Section: Executor lifecycle

If code creates an ExecutorService, it owns its lifecycle unless ownership is explicitly transferred. Shut it down appropriately and handle interruption by restoring the interrupt status when catching InterruptedException and not propagating it.

```
try {
    performInterruptibleWork();
} catch (InterruptedException ex) {
    Thread.currentThread().interrupt();
    throw new IllegalStateException("Work was interrupted", ex);
}
```

Heading: Directory and Package Structure

Section: General Java layout

```
src/
  main/
    java/
      com/example/app/
        domain/
        service/
        repository/
        api/
        config/
    resources/
      application.yml
  test/
    java/
      com/example/app/
        domain/
        service/
        repository/
        api/
    resources/
```

This is a starting point, not a mandate. A simple library or command-line program may need far fewer packages. A Spring application may use feature-oriented packages such as billing, identity, or inventory, with layers inside each feature. Follow the existing structure unless the task requires a structural change.

Section: Package naming and dependency direction

- MUST use lowercase package names and avoid names that conflict with java.* or other platform namespaces.
- SHOULD organize packages around cohesive features or layers, depending on project scale and existing architecture.
- SHOULD keep domain rules independent of transport and persistence details where practical.
- MUST avoid circular package dependencies.
- SHOULD keep implementation classes package-private when no external package needs them.
- MUST not move large parts of a repository merely to match a sample layout.

Section: Tests mirror production code

Place tests under src/test/java using package names that mirror production packages. This makes related tests easy to find and often permits package-private access when justified. Test names should describe behavior rather than mirror the production method name mechanically.

Heading: Dependency-Selection Guidance

Section: Inspect before adding

The Java standard library already provides collections, streams, date/time APIs, concurrency primitives, logging interfaces in many application stacks, file APIs, and testing-independent utilities. Before adding a dependency, inspect pom.xml or the project's build file, dependency-management rules, existing utility packages, and the version constraints.

- MUST inspect existing dependencies and versions before proposing a new library.
- MUST explain the capability needed and why existing APIs are insufficient.
- SHOULD prefer maintained, well-established libraries with a clear compatibility story.
- MUST not hard-code versions that conflict with a parent POM or dependency-management platform.
- SHOULD consider transitive dependencies, vulnerability posture, license requirements, startup cost, and operational support.
- MAY add a dependency when it materially reduces risk or complexity and the project accepts it.

Section: Common library categories

Need	Possible choice	Selection guidance
Unit/integration tests	JUnit 5, Mockito, Spring Boot test starter	Use what the project already manages; avoid redundant test frameworks.
Logging in a managed application	SLF4J API with the project's backend	Do not add a second logging backend without a deliberate reason.
Bean validation	Jakarta Bean Validation provider	Needed only when using that validation model; framework starters may supply it.
JSON mapping	Existing framework mapper, often Jackson	Reuse the application's configured mapper and conventions.
Static analysis / formatting	Existing formatter, Checkstyle, SpotBugs, or equivalent	Adopt only when the repository or team uses it; do not add all tools by default.

Section: Dependency decision checklist

- What concrete problem does the dependency solve?
- Can the JDK or an existing dependency solve it clearly and safely?
- Is it compatible with the project's Java version and framework versions?
- Is it already supplied transitively or through a starter?

- Does the team have an approved version and update process?
- Will it require configuration, runtime services, or operational ownership?

Heading: Repeatable Requirement Implementation Workflow

Section: Step 1 — Understand the requirement

- Restate the requested behavior and identify observable acceptance criteria.
- Identify affected APIs, data models, business rules, side effects, and error cases.
- Separate explicit requirements from assumptions; ask for clarification when an ambiguity changes behavior materially.
- Check whether backward compatibility is required.

Section: Step 2 — Inspect the repository

- Read the relevant source files and nearby tests.
- Inspect the build configuration, Java release, package structure, formatter, and static-analysis rules.
- Find similar implementations and follow their established conventions when appropriate.
- Check whether the requested functionality already exists before creating another implementation.

Section: Step 3 — Plan the smallest coherent change

Pseudocode:

```
INPUT: requirement, repository context
1. Identify acceptance criteria and constraints.
2. Locate the owning class/module and existing behavior.
3. List code, tests, and configuration that must change.
4. Select the simplest design consistent with current architecture.
5. Identify validation, failure, logging, and compatibility needs.
6. Define tests for success, failure, and relevant boundaries.
7. Implement only the required scope.
8. Run focused tests, then broader checks when practical.
9. Review diff for accidental changes, secrets, and missing cases.
10. Report changes, tests run, and any unresolved limitations.
```

Section: Step 4 — Implement in layers

Keep responsibilities clear. Parse and validate inputs at boundaries; put domain decisions in appropriate domain/service code; keep persistence and transport details in their respective layers. Avoid duplicating validation logic across layers unless each layer protects a distinct invariant.

Section: Step 5 — Verify

- Compile with the project's configured Java release.
- Run focused tests for the changed behavior.
- Run relevant broader tests or static checks when time and environment permit.
- Inspect the final diff for unrelated formatting, generated files, credentials, debug output, and unintended API changes.
- If tests cannot run, say so explicitly and state what verification was performed instead.

Section: Step 6 — Report

A useful implementation report states: what changed, why it satisfies the acceptance criteria, files or components affected, tests and commands run, and remaining risks or assumptions. Never claim a test passed unless it was actually executed and passed.

Heading: Incorrect Versus Corrected Implementations

Section: Example: exposing mutable state

```
// Incorrect: callers can change internal state without validation.
public class Playlist {
    public List&lt;String&gt; songs = new ArrayList&lt;&gt;();
}

// Corrected: own the state and expose a safe view.
public final class Playlist {
    private final List&lt;String&gt; songs = new ArrayList&lt;&gt;();

    public void addSong(String song) {
        songs.add(Objects.requireNonNull(song, "song"));
    }

    public List&lt;String&gt; songs() {
        return List.copyOf(songs);
    }
}
```

The corrected version prevents callers from mutating the internal list directly. If the class is accessed concurrently, additional synchronization or a different ownership model may still be necessary.

Section: Example: broad exception catch

```
// Incorrect: loses context and hides the failure.
try {
    return parseAmount(raw);
} catch (Exception ex) {
    return BigDecimal.ZERO;
}

// Corrected: reject invalid input or translate at the boundary.
try {
    return new BigDecimal(raw);
} catch (NumberFormatException ex) {
    throw new IllegalArgumentException("amount must be numeric", ex);
}
```

Returning zero for invalid input is only correct if the domain explicitly defines invalid amounts as zero. Otherwise it converts a failure into plausible but incorrect data.

Section: Example: stream side effects

```
// Avoid: side effects hidden inside a transformation.
users.stream()
    .map(user -&gt; {
        auditService.record(user);
        return user.activate();
    })
    .toList();

// Prefer an explicit loop for side-effecting commands.
for (User user : users) {
    auditService.record(user);
    user.activate();
}
```

Section: Example: unclear boolean and magic value

```
// Incorrectly hides the meaning of both arguments.
schedule(task, true, 5000);

// Prefer named concepts.
schedule(task, ScheduleMode.RETRY, Duration.ofSeconds(5));
```

The exact enum or API shape depends on the application. The standard is to make behavior and units clear at the call site.

Section: Example: null collection

```
// Incorrect: callers must remember to check for null.
public List&lt;Order&gt; findOpenOrders() {
    if (noOrdersExist()) {
        return null;
    }
    return loadOpenOrders();
}

// Corrected: empty collection represents no results.
public List&lt;Order&gt; findOpenOrders() {
    if (noOrdersExist()) {
        return List.of();
    }
    return loadOpenOrders();
}
```

Section: Example: unnecessary inheritance

```
// Avoid inheriting solely to reuse implementation.
class EmailReportService extends EmailSender {
    ...
}

// Prefer an explicit collaborator.
final class EmailReportService {
    private final EmailSender emailSender;

    EmailReportService(EmailSender emailSender) {
        this.emailSender = emailSender;
    }
}
```

Heading: Rules for Research and Coding Agents

Section: Research Agent responsibilities

- MUST distinguish repository facts from general recommendations.
- MUST cite or identify the source file and relevant symbol when describing existing behavior, if available in the task context.
- SHOULD retrieve only knowledge-base sections relevant to the requirement rather than passing an entire reference document when a focused chunk suffices.
- MUST flag uncertainty when code, tests, or configuration do not establish a claim.
- MUST not treat comments or retrieved documentation as proof that the current implementation behaves as described; inspect implementation and tests.

Section: Coding Agent responsibilities

- MUST use the project's actual Java release and dependencies rather than assuming the latest versions.
- MUST preserve existing behavior outside the requested scope.
- MUST not create new dependencies, packages, abstractions, or configuration without a task-related reason.
- MUST include or update tests proportionate to the change.
- MUST not invent missing business rules to make code compile.
- MUST report commands run and their actual outcomes.
- SHOULD prefer a minimal patch that is easy for a human reviewer to understand.

Section: Knowledge retrieval guidance

For RAG retrieval, each topic section should be understandable independently. The agent should combine the retrieved rule with repository evidence and the task's acceptance criteria. If the reference says “SHOULD prefer composition,” that is guidance, not an instruction to refactor every inheritance relationship in the repository.

Conflict resolution: Follow explicit task requirements and safety constraints first, then project-specific architecture and conventions, then applicable MUST rules, then SHOULD recommendations. Record justified deviations when they materially affect maintainability or risk.

Heading: Java Code-Review Checklist

Section: Correctness and scope

- The change satisfies the stated acceptance criteria.
- Existing public contracts and unrelated behavior are preserved.
- Boundary cases, invalid inputs, and expected failure paths are handled.
- No invented business rule or silent fallback changes the meaning of data.

Section: Design and readability

- Names reveal intent and use consistent domain terminology.
- Classes and methods have cohesive responsibilities.
- Encapsulation protects invariants and mutable state.
- Abstractions are justified; composition is preferred where appropriate.
- Control flow is understandable and method side effects are visible.

Section: Java correctness

- No raw types or avoidable unchecked casts.
- Optional values and nullable inputs are handled deliberately.
- Collections match ordering, uniqueness, lookup, and concurrency requirements.
- equals and hashCode are consistent where required.
- Resources are closed by their owner, and exceptions preserve useful context.
- Concurrency assumptions and shared mutable state are addressed.

Section: Operational quality

- Logs are useful and do not expose secrets or unnecessary personal data.
- No credentials, tokens, or local machine-specific paths are committed.
- New dependencies are justified and compatible with the existing build.
- Tests cover the changed behavior proportionately and are deterministic.
- Formatting and static checks follow repository conventions.

Section: Final agent self-check

```
Before finalizing a code change, answer:
1. What requirement does each changed block satisfy?
2. What existing behavior might this affect?
3. What invalid or boundary input could break it?
4. What evidence supports the implementation assumptions?
5. Which tests were run, and what were their actual results?
6. Did I add unnecessary abstractions, dependencies, or unrelated edits?
7. Can a human reviewer understand the diff without reconstructing my reasoning?
```

End of PDF 1 — Java Coding Standards. This document is a general Java 21 reference. For framework-specific REST, security, persistence, and testing guidance, use the corresponding DevPilot AI knowledge-base documents in the prescribed sequence.

Document filename: `java-coding-standards.pdf`